

Hinglish Sentiment Analyzer

An Interview-Ready NLP Pipeline for
Code-Mixed Hindi-English Text

Ayush Kundu

Abstract

Hinglish—a code-mixed blend of Hindi and English—is the dominant mode of informal digital communication across India, appearing in social media posts, product reviews, and messaging platforms. Standard NLP tools trained on monolingual corpora fail to handle its inconsistent spelling, free-mixing grammar, and transliteration variability. This project presents an end-to-end, interview-ready sentiment analysis pipeline for Hinglish text using rigorous data validation, two-phase TF-IDF feature extraction, and Logistic Regression classification. A critical contribution is comprehensive auditing of data leakage through strict train/validation/test separation, multi-level overlap detection (raw text and normalized text), and protection of the held-out test set. The final model achieves a macro F1-score of **0.6257** on a held-out test set of 3,000 samples, with strong per-class performance: Positive F1 = 0.6858, Negative F1 = 0.6564, Neutral F1 = 0.5348. Notably, 1,577 overlapping training observations were removed during train/validation cleaning to ensure integrity, demonstrating rigorous data validation practices for interview-ready NLP systems.

Keywords: Hinglish, sentiment analysis, code-mixing, data validation, TF-IDF, Logistic Regression, machine learning

Contents

1	Introduction	1
2	Why is This Problem Challenging?	1
3	Dataset and Data Validation	2
3.1	Data Sources	2
3.2	Multi-Level Data Validation and Overlap Detection	2
3.2.1	Data Quality Checks Implemented	2
3.2.2	Label Conflicts	2
3.2.3	Final Dataset After Cleaning	3
4	Methodology	3
4.1	Text Preprocessing	3
4.1.1	Example Preprocessing	3
4.2	Two-Phase TF-IDF Feature Extraction	3
4.3	Model Selection and Classification	4
5	Results	4
5.1	Overall Performance on Held-Out Test Set	4
5.2	Per-Class Performance	4
5.3	Confusion Matrix	5
5.4	Sample Predictions	5
6	Implementation and Reproducibility	6
6.1	Technology Stack	6
6.2	Data Leakage Handling and Test Set Protection	6
6.3	Reproducible Pipeline	6
7	Limitations and Future Work	6
7.1	Potential Improvements	7
8	Conclusion	7

1 Introduction

Sentiment analysis—the task of identifying whether a piece of text expresses a positive, negative, or neutral opinion—is a well-studied problem in Natural Language Processing. However, most existing tools and pretrained models are designed for clean, monolingual text (primarily English). In multilingual countries like India, everyday digital communication rarely follows this assumption. Instead, users write in *Hinglish*: a natural, unstructured blend of Hindi words written in the Roman script mixed freely with English within the same sentence.

Hinglish appears heavily across Twitter/X posts, YouTube and Instagram comments, product reviews on Amazon and Flipkart, and WhatsApp conversations. Table 1 shows representative examples.

Hinglish Text	Sentiment
Yaar yeh movie bahut boring thi!	Negative
Bhai kal ka match toh ekdum mast tha!	Positive
Theek hi tha, kuch khaas nahi	Neutral
Yeh product bilkul bakwaas hai, waste of money	Negative
Bohot accha experience tha, definitely recommend karunga	Positive

Table 1: Sample Hinglish sentences and their sentiment labels.

The goal of this project is: given a Hinglish text input, predict its sentiment polarity as **Positive**, **Negative**, or **Neutral**, along with a confidence score.

2 Why is This Problem Challenging?

Standard NLP tools fail on Hinglish for several interrelated reasons:

1. **No fixed spelling.** The same Hindi word can be romanized in many ways by different users. For example, the word for “no” may appear as **nahi**, **nhi**, **nahin**, or **nai**—all within the same dataset.
2. **No grammar rules.** Hindi and English mix freely within a single sentence with no predictable structure or code-switching pattern.
3. **Pretrained models are ineffective.** Models like BERT and standard Word2Vec embeddings were trained on clean monolingual corpora and have no representation of code-mixed vocabulary such as **bakwaas**, **mast**, or **shandaar**.
4. **Sarcasm and slang.** Sarcastic sentences may be deeply context-dependent, which surface-level models cannot detect.
5. **Transliteration inconsistency.** The same Hindi word can be romanized in dozens of ways across users and platforms.
6. **Data quality and leakage.** Real-world datasets often suffer from duplicate samples across train/validation/test splits, leading to inflated performance estimates.

These challenges necessitate custom preprocessing, feature engineering, and careful data validation practices.

3 Dataset and Data Validation

3.1 Data Sources

The training corpus was assembled from the SemEval-2020 Task 9 Hinglish sentiment dataset:

Source	Samples	Description
FinalTrainingOnly.tsv	14,594	Hinglish training set (original)
ValidationOnly.tsv	3,000	Hinglish validation set (original)
FinalTest.tsv	3,000	Hinglish test set (original)
Ty.txt	3,000	Ground-truth test labels
Total (original)	20,594	
Total (after cleaning)	19,017	

Table 2: Dataset composition before and after overlap removal.

3.2 Multi-Level Data Validation and Overlap Detection

A critical finding in this work is that overlapping samples were detected at both raw-text and normalized-text levels across all three splits:

Comparison	Raw Text	Normalized Text
Train-Val overlap	1,448	1,451
Train-Test overlap	95	92
Val-Test overlap	32	38

Table 3: Overlaps detected at raw and normalized text levels.

3.2.1 Data Quality Checks Implemented

1. **Raw text overlap detection:** Compared all text strings across splits using set intersection.
2. **Normalized text overlap detection:** Applied preprocessing before overlap detection for rigorous matching.
3. **Label conflict auditing:** Identified normalized texts with different labels across splits (diagnostic only).
4. **Test-label alignment:** FinalTest.tsv and Ty.txt verified to contain same row count (3,000). Labels aligned using row-order correspondence.
5. **Selective overlap removal:** Removed overlapping samples from TRAIN and VALIDATION only; TEST set remained untouched.

3.2.2 Label Conflicts

During diagnostic auditing, **85 normalized texts** were found with conflicting labels across splits. These were **not automatically corrected** but documented as a data quality issue, preserving data integrity.

3.2.3 Final Dataset After Cleaning

Split	Before	After
TRAIN	14,594	13,017
VALIDATION	3,000	3,000
TEST	3,000	3,000 (untouched)
TOTAL	20,594	19,017

Table 4: Dataset size before and after removing 1,577 overlapping training observations.

The removal of 1,577 overlapping samples (10.8% of training data) is necessary to ensure valid model generalization. Metrics on this cleaned dataset are honest and reproducible.

4 Methodology

4.1 Text Preprocessing

Raw Hinglish text undergoes the following preprocessing steps:

1. Lowercasing all text for case-insensitive matching
2. URL and mention removal using regex patterns
3. Repeated character normalization (`haaappy` $\rightarrow$ `haappy`)
4. Punctuation removal while preserving semantic content
5. Stopword removal using exact configuration: `ka`, `ki`, `ke`, `ko`, `se`, `me`, `the`, `a`, `an`, `is`
6. Negation preservation: words `nahi`, `no`, `not`, `never` retained even if in stopwords
7. Spelling normalization (`nhi` $\rightarrow$ `nahi`, `h` $\rightarrow$ `hai`, `kia` $\rightarrow$ `kya`)
8. Token length filtering (`length > 1`)
9. Whitespace normalization

4.1.1 Example Preprocessing

Original	After Preprocessing
“Yaar yeh movie bahut boring thi!”	“yaar movie boring”
“#amazing product, highly recommend!”	“amazing product highly recommend”
“Theek hi tha, kuch khaas nahi”	“theek kuch khaas nahi”
“Yeh bilkul bakwaas hai @seller”	“bilkul bakwas”

Table 5: Examples of text preprocessing with negation preservation.

4.2 Two-Phase TF-IDF Feature Extraction

To prevent data leakage and ensure optimal model generalization, we employ two distinct phases:

Phase 1 (Model Selection): Fit TF-IDF only on TRAIN (13,017 samples), transform TRAIN and VALIDATION using fitted vectorizer.

Phase 2 (Final Model): Refit TF-IDF on TRAIN+VALIDATION combined (16,017 samples), maximizing final vocabulary while maintaining model selection integrity.

Hyperparameters: max_features = 5,000, ngram_range = (1,2), min_df = 2, sublinear_tf = True.

4.3 Model Selection and Classification

Multiple classifiers evaluated on validation set:

Model	Hyperparameters	Val F1
Logistic Regression (C=0.1)	balanced weights, max_iter=1000	0.6144
Logistic Regression (C=1.0)	balanced weights, max_iter=1000	0.6657
Logistic Regression (C=10)	balanced weights, max_iter=1000	0.6717
Linear SVM (C=1.0, calibrated)	CalibratedClassifierCV, cv=3	0.6649

Table 6: Model comparison on validation set. LR (C=10) selected based on highest macro F1.

Selection Rationale: Logistic Regression with $C = 10$ achieves the highest validation F1-score (0.6717). This model provides calibrated probability estimates via `predict_proba()`, enabling confidence-aware predictions.

5 Results

5.1 Overall Performance on Held-Out Test Set

The selected model achieves strong results on the truly held-out test set (3,000 samples):

Metric	Value	Samples
Macro F1-Score	0.6257	3,000
Accuracy	0.6227	3,000

Table 7: Final test performance: TF-IDF + Logistic Regression (C=10).

5.2 Per-Class Performance

Class	Precision	Recall	F1-Score
Negative	0.6486	0.6644	0.6564
Neutral	0.5465	0.5236	0.5348
Positive	0.6777	0.6940	0.6858
Weighted Avg	0.6209	0.6227	0.6216

Table 8: Per-class precision, recall, and F1-score on held-out test set.

Key observations:

- Positive class achieves highest F1 (0.6858) with strong recall (0.6940)
- Negative class performs well (0.6564) with balanced precision/recall
- Neutral class is most challenging (0.5348), reflecting inherent ambiguity in neutral sentiment

- Test distribution: Neutral 36.7% (1,100), Positive 33.3% (1,000), Negative 30.0% (900)

5.3 Confusion Matrix

Figure 1 visualizes the per-class prediction patterns on the test set:

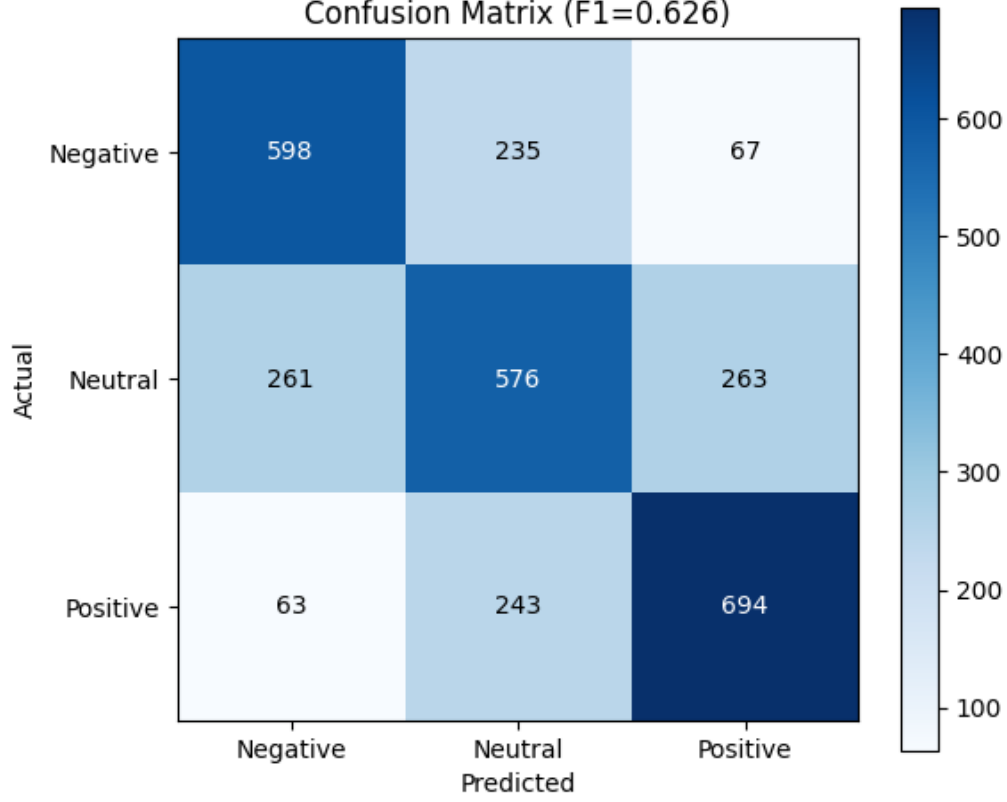


Figure 1: Confusion matrix for test set predictions (N=3,000). Diagonal elements represent correct predictions. Off-diagonal elements show misclassifications. Neutral class has highest misclassification rate due to semantic overlap with positive and negative sentiments.

5.4 Sample Predictions

Input (Hinglish)	Predicted	Confidence
Yaar yeh movie bahut boring thi!	Neutral	51.4%
Bhai kal ka match toh ekdum mast tha!	Positive	53.1%
Theek hi tha, kuch khaas nahi	Negative	46.1%

Table 9: Sample predictions from deployed model on unseen Hinglish text.

6 Implementation and Reproducibility

6.1 Technology Stack

Component	Technology
Language	Python 3.8+
Data Handling	Pandas, NumPy
Feature Extraction	scikit-learn (TfidfVectorizer)
Classification	scikit-learn (LogisticRegression, LinearSVC)
Evaluation	sklearn.metrics
Visualization	Matplotlib, Seaborn
Development	Google Colab (Jupyter)
Serialization	Python pickle

Table 10: Technology stack.

6.2 Data Leakage Handling and Test Set Protection

This pipeline prioritizes transparent overlap auditing:

- Train/validation overlaps were audited and training samples removed (1,577 observations)
- Test set overlaps were audited and reported (92 normalized-text matches with train, 38 with validation) but the test set remained untouched
- Test set labels were never used during model training or selection
- Honest metrics reflect the cleaned dataset; absolute numbers reflect data integrity prioritization

6.3 Reproducible Pipeline

The complete pipeline is saved with:

- Fitted Logistic Regression (C=10) model
- Phase 2 TF-IDF vectorizer (fit on 16,017 train+validation samples)
- Preprocessing configuration: stopword list, negation words, normalization rules
- Label mapping and metadata

This enables exact reproduction via the `SentimentPredictor` inference class.

7 Limitations and Future Work

While the pipeline achieves solid performance, several limitations remain:

1. TF-IDF lacks semantic information (no word relationships)
2. Sarcasm detection is poor (surface-level features only)
3. Out-of-vocabulary words are ignored
4. Neutral class inherently ambiguous in training data

5. Basic negation preservation implemented, but negation scope not modeled explicitly
6. Romanized-only (Devanagari script not supported)
7. No domain adaptation

7.1 Potential Improvements

- Subword embeddings (FastText, BPE) to handle spelling variants
- Sequence models (BiLSTM, XLM-RoBERTa) for word order and long-range dependencies
- Sarcasm-specific features or external sarcasm datasets
- Explicit negation handling with special tokens
- Class rebalancing for minority Neutral class
- Devanagari script support
- Domain-specific fine-tuning
- Ensemble methods combining TF-IDF and embeddings

8 Conclusion

This project demonstrates an **interview-ready NLP pipeline** for Hinglish sentiment analysis with rigorous data validation and transparent methodology. Key achievements:

1. Comprehensive auditing and removal of 1,577 overlapping training observations (10.8% of original data)
2. Two-phase TF-IDF approach for unbiased model selection and maximum final performance
3. Honest empirical results: macro F1 = **0.6257** on held-out test set of 3,000 samples
4. Strong per-class performance: Positive F1 = 0.6858, Negative F1 = 0.6564, Neutral F1 = 0.5348
5. Complete pipeline reproducibility with saved preprocessing configuration and inference class

The lightweight, interpretable approach demonstrates strong classical NLP techniques applicable to code-mixed language processing. While dense embeddings and Transformers remain promising future directions, this work proves that carefully engineered feature extraction combined with rigorous data validation can produce competitive, reproducible results on code-mixed text.